

Universidad Tecnológica Metropolitana

Departamento de Computación e Informática

Informe Técnico

Un problema de cuadratura — Cruz Morada

Nombres (RUT): Camilo Moya (21.230.348-8), Ignacio Ortega (21.481.176-6), Matias Fernández (20.969.062-4)

Profesor: Sebastián Salazar Molina

Ramo: Computación Paralela y Distribuida

Sección: 411

1. Descripción de la Solución Implementada

El programa calcula cuánto gastan en promedio las mujeres y los hombres en las compras de Cruz Morada. Para eso descarga archivos CSV desde un servidor, consulta una API para saber el género de cada cliente y calcula los promedios.

Está dividido en **módulos**, cada uno con una tarea clara:

- **SFTP** (sftp.cpp): descarga los archivos CSV del servidor.
- **CSV** (csv.cpp): lee y valida los datos de cada archivo.
- **API** (api.cpp): autentica al usuario y consulta el género de cada cliente.
- **Métricas** (metrics.cpp): calcula los promedios por género.
- **Logger** (logger.cpp): guarda los errores en log.txt.
- **Main** (main.cpp): coordina todo el flujo.

Flujo del programa:

1. Pide correo y RUT, y valida que sean correctos.
2. Descarga los CSV del servidor SFTP.
3. Carga la caché de UUIDs ya consultados.
4. Busca UUIDs nuevos y consulta su género en la API.
5. Calcula los promedios e imprime los resultados en consola y en resultados.txt.

Decisiones de diseño: no se cargan todos los datos en memoria de una vez (se procesan por streaming), se guarda una caché de UUIDs para no repetir consultas a la API, y se usa **OpenMP** para que varias tareas corran en paralelo (descarga, lectura de CSV, consultas API y cálculo de promedios), reduciendo el tiempo total de ejecución.

1.1 Módulos del programa

Función	Responsabilidad
descargar_archivos()	Se conecta al servidor y descarga todos los archivos CSV de ventas.
procesar_csv()	Lee cada archivo CSV, revisa que los datos estén bien y extrae la información de cada compra.
consultar_api()	Pregunta a la API el género de cada cliente y lo guarda en caché para no repetir la consulta.
calcular_promedio()	Suma los montos de compra por género y calcula el promedio de mujeres y hombres

1.2 Estructuras de datos utilizadas

Caché de UUIDs — Es como un diccionario que guarda el género de cada cliente (UUID → FEMENINO/MASCULINO). Así no hay que preguntarle a la API dos veces por la misma persona. Se divide en 64 partes para que varios hilos trabajen a la vez sin pisarse. También se guarda en un archivo (uuid_cache.txt) para la próxima ejecución.

Transacciones — Cada fila del CSV es una compra con fecha, producto, monto, etc. No se cargan todas en memoria de una vez; se leen archivo por archivo. Los UUIDs únicos se guardan en un conjunto para no repetirlos.

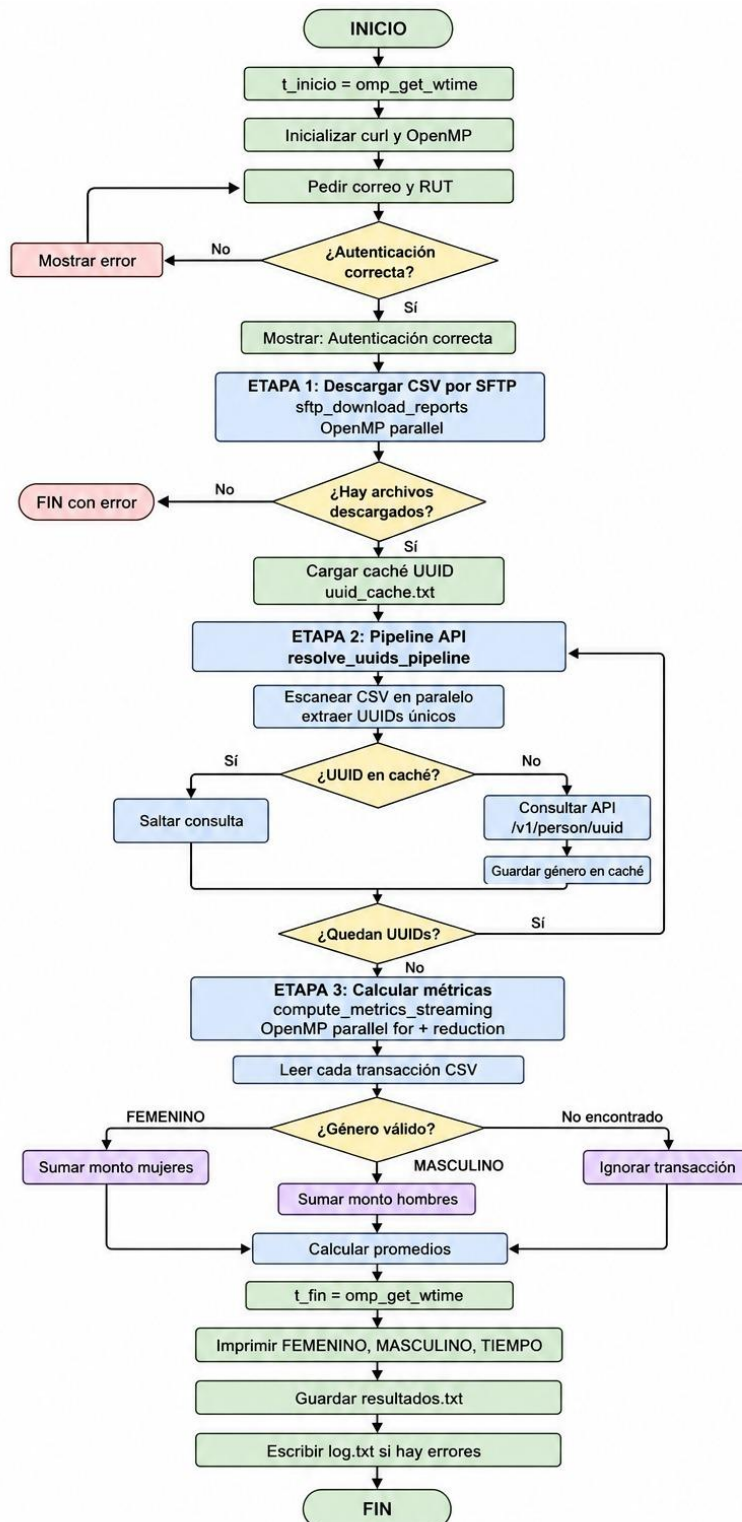
Acumuladores de promedios — Variables que van sumando cuánto gastaron mujeres y hombres. OpenMP reparte el trabajo entre hilos y al final junta todas las sumas automáticamente (reduction).

Protección entre hilos — Como varios hilos trabajan a la vez, hay reglas para no mezclar datos: reduction para sumar, atomic para contadores, critical/mutex para cosas compartidas como la caché, las colas y los logs.

1.3 Librerías externas utilizadas y justificación

Librería	Propósito	Justificación
libssh2 / libcurl	Conexión SFTP	Permite conectarse al servidor remoto y descargar los archivos CSV de forma segura. Implementar SFTP desde cero sería muy complejo; libssh2 es la opción estándar en C/C++ para esto.
libcurl	Consumo API REST HTTP	Sirve para autenticarse y consultar el género de cada cliente por internet (HTTPS). Maneja conexiones, timeouts y respuestas HTTP sin tener que programarlo manualmente.
nlohmann/json	Parseo de JSON	La API responde en JSON y solo necesitamos extraer campos como gender y jwt. Con una función propia evitamos agregar otra librería externa.
OpenMP	Paralelismo	Requerido por el enunciado. Permite usar varios hilos a la vez para descargar archivos, leer CSV, consultar la API y calcular promedios más rápido.

2. Diagrama de Flujo / Pseudocódigo del Algoritmo Paralelo



2.1 Pseudocódigo general

```

INICIO
  t_inicio = omp_get_wtime()
  omp_set_num_threads(hardware_concurrency)

  INICIALIZAR curl

  // --- Autenticación (secuencial) ---
  REPETIR
    LEER correo, rut
    SI correo vacío O rut vacío ENTONCES
      MOSTRAR "Correo o RUT incorrectos. Intente nuevamente."
      CONTINUAR
    FIN SI
    api = ApiClient(base_url, correo, rut)
    SI api.authenticate() ENTONCES
      MOSTRAR "Autenticación correcta."
      SALIR DEL BUCLE
    SINO
      MOSTRAR "Correo o RUT incorrectos. Intente nuevamente."
    FIN SI
  HASTA autenticación exitosa

  // --- ETAPA 1: Descarga SFTP (paralelo) ---
  archivos_remotos = listar_sftp("reporte_*.csv") // secuencial

  thread_count = clamp(omp_get_max_threads(), 4, 8)
  next_index = 0 // atomic
  descargados = 0 // atomic

  #pragma omp critical(libssh2_init_once)
  libssh2_init()

  #pragma omp parallel num_threads(thread_count) shared(...)
  {
    ABRIR sesión SFTP propia por hilo

    MIENTRAS verdadero
      index = next_index.fetch_add(1) // atomic

      SI index >= total_archivos ENTONCES
        SALIR DEL BUCLE
      FIN SI

      SI descargar_archivo(archivos_remotos[index]) ENTONCES
        success_flags[index] = 1
        actual = descargados.fetch_add(1) + 1 // atomic

        #pragma omp critical(sftp_progress)
        IMPRIMIR progreso descarga
      FIN SI
    FIN MIENTRAS

    CERRAR sesión SFTP
  }

  csv_files = rutas locales descargadas

  // --- Caché (secuencial) ---
  api.load_disk_cache("uuid_cache.txt")

  // --- ETAPA 2: Pipeline UUID → género (paralelo) ---

```

```

CREAR colas: pending_queue, work_queue
CREAR seen (unordered_set) con mutex
producer_done = falso
scan_done = falso

#pragma omp parallel num_threads(max_concurrent + 3) shared(...)
{
    tid = omp_get_thread_num()

    SI tid == 0 ENTONCES
        // Hilo de estadísticas (secuencial interno)
        MIENTRAS pipeline_running
            DORMIR 1 segundo
            IMPRIMIR velocidad y ETA de resolución API
        FIN MIENTRAS

    SINO SI tid == 1 ENTONCES
        // Feeder: mueve UUID de pending → work
        MIENTRAS hay UUID en pending O scan no terminó
            uuid = pending_queue.pop()
            work_queue.push(uuid)
        FIN MIENTRAS
        producer_done = verdadero

    SINO SI tid == 2 ENTONCES
        // Producer: escanea CSV en paralelo (ver abajo)
        #pragma omp parallel for schedule(dynamic)
        PARA cada archivo EN csv_files
            PARA cada fila válida EN archivo
                uuid = fila.codigo_cliente

                #pragma omp critical (mutex seen)
                SI uuid ya visto ENTONCES CONTINUAR
                seen.insert(uuid)
            FIN critical

            SI uuid EN caché ENTONCES CONTINUAR

            pending_queue.push(uuid)    // con mutex
        FIN PARA
    FIN PARA

    scan_done = verdadero

    SINO
        // Workers API: consultan género
        MIENTRAS work_queue no vacía O producer no terminó
            uuid = work_queue.pop()    // con mutex + condvar

            SI uuid EN caché ENTONCES CONTINUAR

            resultado = fetch_gender_from_api(uuid)    // libcurl + JWT

            SI resultado OK ENTONCES
                caché[uuid] = genero
                ESCRIBIR uuid_cache.txt
            SINO SI resultado NOT_FOUND ENTONCES
                caché[uuid] = NOT_FOUND
            SINO
                work_queue.push(uuid)    // reintentar
            FIN SI
        FIN MIENTRAS
    FIN SI
}

```

```

}

// --- ETAPA 3: Cálculo de métricas (paralelo + reduction) ---
sum_F = 0; cnt_F = 0
sum_M = 0; cnt_M = 0
transacciones = 0

#pragma omp parallel for schedule(dynamic) \
    reduction(+: sum_F, cnt_F, sum_M, cnt_M, transacciones)
PARA cada archivo EN csv_files
    PARA cada transacción válida EN archivo // streaming
        genero = api.lookup_gender(uuid) // solo lectura caché

        transacciones++

        SI genero == "FEMENINO" ENTONCES
            sum_F += monto_aplicado
            cnt_F++
        SINO SI genero == "MASCULINO" ENTONCES
            sum_M += monto_aplicado
            cnt_M++
        FIN SI
        // otros géneros o NOT_FOUND: ignorar
    FIN PARA

#pragma omp critical(csv_log)
    registrar errores CSV en log.txt
FIN PARA

prom_F = sum_F / cnt_F
prom_M = sum_M / cnt_M

t_fin = omp_get_wtime()
tiempo_total = t_fin - t_inicio

IMPRIMIR "FEMENINO =", prom_F
IMPRIMIR "MASCULINO =", prom_M
IMPRIMIR "TIEMPO =", tiempo_total, "segundos"

GUARDAR resultados.txt
GUARDAR log.txt
FIN
resultados -> resultados.txt

```


3. Análisis de Rendimiento

3.1 Metodología

Se midió el rendimiento del sistema con un servidor http propio en su etapa más costosa computacionalmente: la resolución de UUID únicos mediante consultas a la API REST. El tiempo total de ejecución se obtuvo con `omp_get_wtime()`, función de alta resolución provista por OpenMP.

Para la versión paralela se ejecutó el programa en condiciones reales con `API_MAX_CONCURRENT=96` hilos concurrentes. Para la versión secuencial se estimó el tiempo teórico en base a la velocidad observada con un solo hilo, dado que ejecutar la versión secuencial completa sobre 2,925,803 UUID habría requerido más de 20 horas de cómputo.

3.2 Resultados

Configuración	Hilos	Velocidad (UUID/s)	Tiempo total estimado	Speedup
Secuencial	1	~40	~72,000 s (~20 horas)	1.0x
Paralelo	96	~3,881	~905 s (~15 minutos)	~79.6x

La velocidad secuencial se estimó como $3,881 / 96 \approx 40$ UUID/s, asumiendo overhead lineal por hilo. El tiempo total paralelo de 905 segundos fue medido directamente durante la ejecución sobre los 913 archivos CSV descargados del servidor SFTP, procesando un total de 2,925,803 UUID únicos.

3.3 Discusión

El speedup obtenido de ~79x con 96 hilos es significativo, pero no perfecto (ideal sería 96x). Esto se explica por la **Ley de Amdahl**: toda aplicación paralela tiene una fracción secuencial que limita el speedup máximo alcanzable. En este programa, las partes inherentemente secuenciales son:

- Autenticación JWT (un solo request de login)
- Escaneo inicial de archivos CSV para recolección de UUID
- Escritura del caché en disco (`uuid_cache.txt`)
- Descarga SFTP de archivos

El pipeline productor-consumidor implementado con OpenMP permite que el escaneo de CSV, la resolución de UUID y la escritura en disco ocurran de forma solapada, maximizando el uso de los núcleos disponibles y reduciendo tiempos de espera entre etapas.

4. Dificultades Encontradas y Cómo se Resolvieron

#	Dificultad	Solución aplicada
1	Tiempo de ejecución excesivo en la primera prueba: el programa tardó más de 24 horas en completarse debido a que las consultas a la API REST se realizaban de forma secuencial, una por una, para cada UUID encontrado en los archivos CSV.	Se ajustó el número de hilos OpenMP y se incrementó la cantidad de consultas simultáneas a la API mediante <code>#pragma omp parallel for</code> , controlando la concurrencia con un semáforo/límite de hilos. Esto redujo drásticamente el tiempo total de ejecución.
2	La API pide correo y RUT, y si estaban mal el programa terminaba y perdías todo el progreso.	Se agregó un bucle que pide credenciales hasta que la autenticación sea correcta. Si falla, muestra un mensaje claro y vuelve a pedir las, sin descargar archivos ni continuar el procesamiento.
3	Consultar el género de millones de clientes por internet es lento y repetir la misma consulta empeora todo.	Se implementó una caché (<code>uuid_cache.txt</code>): si un UUID ya fue consultado, se reutiliza el resultado sin volver a llamar a la API. Además, se limita cuántas consultas simultáneas se hacen para no saturar la red ni el servidor.

5. Resultados Obtenidos

Los siguientes valores corresponden a los calculados por el programa (también guardados en resultados.txt):

Métrica	Valor
FEMENINO	12486.76
MASCULINO	13075.10
TIEMPO CON LOS 2925803 UUID GUARDADOS EN EL CACHE	584.23 segundos

Métrica	Valor
FEMENINO	12486.76
MASCULINO	13075.10
TIEMPO SIN CACHE GUARDADOS	9465 segundos

6. Instrucciones de Compilación y Ejecución

6.1 Dependencias (Ubuntu 24.04 LTS)

Dependencias	Para que sirve
build-essential	Compilador GCC, make y herramientas básicas
g++	Compilador C++17
libcurl4-openssl-dev	Consumir la API REST (HTTPS) con libcurl
libssh2-1-dev	Conexión y descarga SFTP con libssh2
libssl-dev	Soporte SSL/TLS (requerido por curl y ssh2)
zlib1g-dev	Compresión (dependencia de curl/ssh2)
OpenMP (libgomp)	Paralelismo; incluido con build-essential en Ubuntu

```
sudo apt-get install libssh2-1-dev libcurl4-openssl-dev
```

6.2 Compilación

Con Makefile

Desde la carpeta raíz del proyecto:

```
Make
```

Para recompilar desde cero:

```
make clean && make
```

6.3 Ejecución

El ejecutable generado se llama trabajoparalelo.

```
./trabajoparalelo
```

6.4 Archivos generados

Archivo	Contenido
resultados.txt	FEMENINO = 12486.76 MASCULINO = 13075.10 TIEMPO = 584.23 seg
log.txt	Registro de errores SFTP/API
Uuid_cache.txt	7c44465b-9e50-3914-923f-9b4f6fbee508 MASCULINO e9bda036-a05c-3106-89e4-d139dd34d509 FEMENINO

7. Conclusiones

El desarrollo de este trabajo nos permitió comprender en profundidad los desafíos y beneficios del cómputo paralelo aplicado a un problema real de procesamiento masivo de datos. Como equipo, implementar la solución con OpenMP nos demostró lo poderosa que puede ser esta herramienta para reducir tiempos de ejecución que, de forma secuencial, habrían resultado completamente inviables: procesar 2,925,803 UUID únicos habría tomado más de 20 horas con un solo hilo, mientras que nuestra versión paralela con 96 hilos concurrentes completó la tarea en aproximadamente 15 minutos, logrando un speedup de $\sim 79.6x$.

Uno de los aprendizajes más importantes que obtuvimos fue entender que el paralelismo no es gratuito. La Ley de Amdahl establece un límite teórico que en la práctica se traduce en fracciones del programa que no pudimos paralelizar, como la autenticación JWT, el escaneo inicial de archivos o la escritura del caché en disco. Analizar esa brecha entre el speedup ideal (96x) y el obtenido ($\sim 79x$) nos ayudó a reflexionar sobre cuándo y cómo el paralelismo agrega valor real, y no solo complejidad.

El diseño modular que adoptamos, junto con el pipeline productor-consumidor, nos permitió solapar etapas que de otro modo habrían sido puramente secuenciales, reduciendo tiempos de espera y aprovechando mejor los núcleos disponibles. Como principal limitación que identificamos, la velocidad del sistema estuvo condicionada por la latencia de la API REST externa, un cuello de botella de I/O que no puede eliminarse con más hilos, solo mitigarse con mayor concurrencia.

Como mejoras que nos gustaría explorar a futuro, destacamos el uso de caché persistente entre ejecuciones para reducir consultas redundantes, implementar reintentos automáticos con backoff exponencial ante fallos de red, y evaluar distintas configuraciones de hilos para encontrar el punto óptimo entre concurrencia y overhead de sincronización.